

ReadMe

Kartengenerierung:

Um eine Karte zu generieren, verwende ich die „`getRandomTerrain()`“-Methode von „`EMapTerrain`“. Diese Methode wählt zufällig verschiedene Terraintypen aus, basierend auf Gewichten, die für jedes Terrain festgelegt sind. Dies stellt sicher, dass wir eine ungefähr richtige Verteilung der verschiedenen Terraintypen auf der Karte haben. Ich habe diese Methode auf [Stackoverflow](#) gefunden und sie implementiert, indem ich sie in einer verschachtelten Schleife mit X- und Y-Indizes für jeden Koordinatenpunkt auf der Karte aufrufe.

Sobald ich die Karte erstellt habe, überprüfe ich sie mit Hilfe von „`MapValidator`“, um sicherzustellen, dass alle Regeln eingehalten werden. „`MapValidator`“ überprüft, ob die Anzahl der Terraintypen auf der Karte stimmt, ob es Inseln gibt (dafür verwende ich den Floodfill-Algorithmus), ob es nicht mehr Wasserfelder an den Kartengrenzen gibt als erlaubt und ob die Kartengröße stimmt. Wenn eine der Regeln verletzt wird, lösche ich die Karte und erstelle eine neue. Dieser Vorgang wird so lange wiederholt, bis die Karte korrekt ist.

Wegfindung:

Meine Strategie besteht darin, alle Grasfelder auf der Karte nacheinander abzusuchen, bis ich den Schatz gefunden habe. Dann wechsele ich zur anderen Hälfte der Karte und suche erneut nach dem Schloss. Zuerst sortiere ich die Grasfelder nach X- und dann nach Y-Koordinaten, um sie nacheinander besuchen zu können (sofern es möglich ist). Wenn ich den Schatz gefunden habe, wechsele ich zur anderen Hälfte der Karte, indem ich das nächstgelegene Feld auf der Karte (basierend auf der Manhattan-Distanz) als Ziel auswähle. Für die Ermittlung von Wegen zu Target-Feldern habe ich den Breadth-First-Search-Algorithmus verwendet.

Netzwerk:

Das Netzwerk ist in der Klasse `Network` implementiert und verwendet die Klassen `ClientServerConverter` und `ServerClientConverter`, um Client-Klassen in Server-Klassen und umgekehrt zu konvertieren. Wenn ein Client beispielsweise eine Bewegung an den Server senden möchte, übersetzt er zuerst die Client `EMapTerrain`-Klasse in die entsprechende `ETerrain`-Klasse des Servers und sendet diese dann. Die Antwort des Servers wird dann mithilfe von `ServerClientController` entsprechend übersetzt.

MVC:

Ich habe das MVC-Pattern in drei Klassen umgesetzt: Der Controller ist die `GameController`-Klasse, die für den Ablauf des Spiels verantwortlich ist und mit anderen Spiellogik-Klassen zusammenarbeitet. Das `GameDataModel` speichert alle notwendigen Spieldaten und aktualisiert die View automatisch mithilfe von `PropertyChangeListener`, einem Observer Pattern in Java. Die View ist die `CLI`-Klasse, die eine Schnittstelle für den Benutzer bereitstellt.